



Universidad Autónoma de Tamaulipas

Producto integrador: punto de venta

Facultad de Ingeniería Tampico

Ingeniería en Sistemas Computacionales

Asignatura: Fundamentos De programación

Grupo: N Grado: 1

Nombre del Docente: Álvarez Navarro Eduardo

Alumno: Guevara Martinez Angel Jeremy

Matricula: 2243330342

```
+ import java.io.BufferedReader;

public class proyecto_integrador {

    static String[][] productos = new String[20][5];

    static String ventas[][];
    static int tamventas = 100;
    static String fecha;

    public static String MostrarMenu(String[] opciones)
    {
        String cadena = "";
        for (String info : opciones)
        {
            cadena = cadena + info + "\n";
        }
        return cadena;
    }

    public static boolean EsNumeroEntero(String dato) {
        for (char c : dato.toCharArray()) {
            if (!Character.isDigit(c)) {
                return false;
            }
        }
        return true;
    }
}
```

Esta parte del código corresponde a la inicialización principal del sistema. Primero se importa la clase `BufferedReader`, la cual permite leer datos ingresados por el usuario desde el teclado, como opciones del menú, códigos de productos o precios. Después se declara la clase principal llamada `proyecto_integrador`, donde se almacenan todas las funciones y variables del programa.

Posteriormente se crea una matriz llamada `productos` con capacidad para almacenar 20 productos y 5 datos por cada uno. Cada fila representa un producto diferente y las columnas guardan el código, nombre, precio, stock y porcentaje de IVA. También se declaran variables relacionadas con las ventas, como la matriz `ventas`, la variable `tamventas` que establece un límite máximo de 100 ventas y la variable `fecha`, que guarda la fecha actual del sistema o del ticket.

La función `MostrarMenu` tiene como objetivo generar el texto de un menú a partir de un arreglo de opciones. La función recorre todas las opciones utilizando un ciclo `for`, las concatena en una sola cadena de texto y agrega saltos de línea para mostrar el menú de manera ordenada al usuario.

Finalmente, la función `EsNumeroEntero` sirve para validar que un dato ingresado sea numérico. La función convierte el texto en un arreglo de caracteres y revisa uno por uno utilizando `Character.isDigit`. Si encuentra un carácter que no sea número, retorna `false`; de lo contrario, retorna `true`. Esta validación ayuda a evitar errores al ingresar datos incorrectos en el sistema.

```
public static boolean EsNumeroDouble(String dato) {
    boolean punto = false;
    if (dato == null || dato.equals("")) {
        return false;
    }
    for (char c : dato.toCharArray()) {
        if (Character.isDigit(c)) {
        } else if (c == '.') {
            if (!punto) {
                punto = true;
            } else {
                return false;
            }
        } else {
            return false;
        }
    }
    return true;
}

public static String LeerValidado(String texto_dialogo, int tipo_dato) throws IOException {
    String cadena = "";
    do {
        cadena = Leer(texto_dialogo);
        if (cadena == null) {
            System.out.println("dato nulo");
        } else {
            if (!EvaluarNumerico(cadena, tipo_dato)) {
                System.out.println("dato incorrecto");
                cadena = null;
            }
        }
    } while (cadena == null);
    return cadena;
}
```

Esta parte del código corresponde a las funciones de validación de números decimales y lectura validada de datos ingresados por el usuario. La función `EsNumeroDouble` tiene como objetivo verificar si un texto puede considerarse un número decimal válido. Primero revisa que el dato no sea nulo ni esté vacío. Después recorre carácter por carácter utilizando un ciclo `for`. Si encuentra números, continúa validando normalmente; si encuentra un punto decimal, verifica que sea el primero, utilizando la variable booleana `punto`. Si aparece un segundo punto o cualquier otro carácter no

permitido, la función retorna false. Si todos los caracteres son válidos, retorna true. Esta función ayuda a validar precios u otros valores con decimales dentro del sistema.

La segunda función llamada LeerValidado sirve para solicitar datos al usuario y asegurarse de que sean correctos antes de continuar con la ejecución del programa. La función recibe dos parámetros: texto_dialogo, que es el mensaje que se mostrará al usuario, y tipo_dato, que indica el tipo de validación que se realizará. Dentro de un ciclo do while, el sistema utiliza la función Leer para capturar el dato ingresado. Si el usuario no introduce información, se muestra el mensaje "dato nulo". Posteriormente, se utiliza la función EvaluarNumerico para validar si el dato corresponde al tipo esperado. Si el dato es incorrecto, el sistema muestra el mensaje "dato incorrecto" y vuelve a solicitar la información hasta que el usuario ingrese un valor válido. Finalmente, la función retorna la cadena validada correctamente.

```
public static boolean EvaluarNumerico(String dato, int tipo)
{
    boolean valido = false;
    switch (tipo) {
        case 1:
            valido = EsNumeroEntero(dato);
            break;

        case 2:
            valido = EsNumeroDouble(dato);
            break;
    }
    return valido;
}

public static String Dialogo(String texto) throws IOException
{
    String cadena;
    System.out.println(texto + " : ");
    BufferedReader lectura = new BufferedReader(new InputStreamReader(System.in));
    cadena = lectura.readLine();
    return cadena;
}

public static String Leer(String texto) throws IOException
{
    String cadena = "";
    cadena = Dialogo(texto);
    if (cadena != null)
    {
        cadena = cadena.trim();
        if (cadena.isEmpty())
            cadena=null;
    }
    else
        cadena = null;

    return cadena;
}
```

Esta parte del código contiene funciones para validar datos numéricos y leer información ingresada por el usuario. La función EvaluarNumerico verifica si un dato es entero o decimal dependiendo del tipo indicado. La función Dialogo

muestra un mensaje y captura texto desde teclado utilizando BufferedReader. Finalmente, la función Leer limpia la cadena ingresada, elimina espacios innecesarios y valida que el dato no esté vacío antes de retornarlo.

```
public static String DesplegarMenu(String Titulo1, String[] menu) throws IOException
{
    String cadena;
    cadena=Titulo1 + "\n" + "\n";
    cadena=cadena+MostrarMenu(menu);
    cadena = cadena + "\n Que opcion deseas ";
    return cadena = Dialogo(cadena);
}

public static String RellenarEspacios(String dato, int tamano)
{
    return String.format("%1$-" + tamano + "s", dato);
}

public static String Fecha() {
    Date fecha = new Date();
    SimpleDateFormat formatodia = new SimpleDateFormat("dd-MM-yyyy");
    return formatodia.format(fecha);
}

public static String IdTicketSiguiente(String idticket) {
    String idticketnext = "";
    int num = Integer.parseInt(idticket) + 1;
    if (num < 10)
    { idticketnext = "00" + String.valueOf(num).trim(); }
    else if ((num > 9) && (num < 100))
    { idticketnext = "0" + String.valueOf(num).trim(); }
    else {
        idticketnext = String.valueOf(num).trim();
    }
    return idticketnext;
}
```

Esta parte del código contiene funciones auxiliares del sistema. La función DesplegarMenu construye y muestra un menú con un título y varias opciones para que el usuario seleccione una acción. La función RellenarEspacios sirve para dar formato al texto y alinear correctamente los datos mostrados en pantalla. La función Fecha obtiene la fecha actual del sistema y la devuelve con formato día-mes-año. Finalmente, la función IdTicketSiguiente genera el siguiente número de ticket incrementando el valor anterior y agregando ceros para mantener un formato ordenado como 001, 002 o 003.

```

• public static int ObtenerUltimaPosicion(String[][] matriz) {
    int ultimaPosicion = -1; // Inicializa la última posición como -1 (indicando que no se encontró una
                             // posición válida)
•   for (int i = 0; i < matriz.length; i++) {
        // Verifica si la primera columna de la fila actual no es nula ni está vacía
•       if (matriz[i][0] != null && !matriz[i][0].isEmpty()) {
            ultimaPosicion = i; // Actualiza la última posición al índice actual
        }
    }
    return ultimaPosicion; // Devuelve la última posición válida encontrada
}

• public static String[][] CargarProductos() {

    String[][] producto = {

        {"001", "Frappe Chocolate", "65", "20", "16"},
        {"002", "Frappe Vainilla", "65", "20", "16"},
        {"003", "Capuchino Combo", "119", "15", "16"},
        {"004", "Latte Combo", "119", "15", "16"},
        {"005", "Lechero Combo", "119", "15", "16"},
        {"006", "Galletas 2x", "80", "25", "16"},
        {"007", "Rebanada de Pastel", "59", "18", "16"},
        {"008", "Tés y Tisanas 2x", "99", "20", "16"},
        {"009", "Panini Tradicional", "125", "10", "16"},
        {"010", "Bisquet con Mantequilla", "89", "15", "16"},

        {"011", "Ramen Coreano Picante", "85", "12", "16"},
        {"012", "Ramen Coreano Queso", "85", "12", "16"},
        {"013", "Ramen Coreano Pollo", "85", "12", "16"},
        {"014", "Ramen Coreano Carne", "85", "12", "16"},
        {"015", "Bebida Fria", "35", "20", "16"},
        {"016", "Sandwich Jamon", "95", "14", "16"},
        {"017", "Croissant", "55", "15", "16"},
        {"018", "Brownie", "45", "15", "16"},
        {"019", "Chocolate Caliente", "58", "18", "16"},
        {"020", "Cafe Americano", "45", "20", "16"}

    };
    return producto;
}

```

La función `ObtenerUltimaPosicion` recorre la matriz de productos y busca la última fila que tenga datos válidos. Esto sirve para identificar hasta qué posición hay información registrada dentro del arreglo.

La función `CargarProductos` crea y carga todos los productos iniciales del sistema en una matriz bidimensional. En esta parte se almacenan los códigos, nombres, precios, existencias e IVA de los productos de la cafetería, permitiendo que el programa tenga disponibles los 20 productos al iniciar el sistema.

```
public static String MostrarProducto(String[] vproducto) {
    String codigo = RellenarEspacios(vproducto[0], 5);
    String nombre = RellenarEspacios(vproducto[1], 30);
    String precio = RellenarEspacios(vproducto[2], 10);
    String cantidad = RellenarEspacios(vproducto[3], 10);
    String iva = RellenarEspacios(vproducto[4], 5);

    String cadena = codigo + nombre + precio + cantidad + iva;
    return cadena;
}

public static String MostrarLista(String[][] vproductos) {
    String salida = "";

    for (int ciclo = 0; ciclo < vproductos.length; ciclo++) {
        String[] vproducto = {
            vproductos[ciclo][0],
            vproductos[ciclo][1],
            vproductos[ciclo][2],
            vproductos[ciclo][3],
            vproductos[ciclo][4]
        };
        String cadena = MostrarProducto(vproducto);
        salida = salida.concat(cadena + "\n");
    }
    return salida;
}

public static int ExisteProducto(String codigo, String[][] vproductos) {
    int enc, pos, tam, ciclo;
    enc = -1;
    pos = 0;
    tam = vproductos.length;
    for (ciclo = 0; ciclo < tam; ciclo++) {
        if (vproductos[ciclo][0].compareTo(codigo.trim()) == 0) {
            enc = pos;
        }
        pos++;
    }
    return enc;
}
```

La función `MostrarProducto` toma la información de un producto y la organiza con espacios para que se muestre de forma ordenada en pantalla. Se encarga de acomodar el código, nombre, precio, cantidad e IVA antes de devolver la cadena completa.

La función `MostrarLista` recorre todos los productos almacenados en la matriz y utiliza la función `MostrarProducto` para mostrar cada producto en formato ordenado. Al final genera una lista completa de todos los productos registrados.

La función `ExisteProducto` verifica si un código de producto ya existe dentro de la matriz de productos. Recorre todos los registros y, si encuentra coincidencia, devuelve la posición del producto; en caso contrario devuelve -1.

```
public static double ObtenerIva(String codigo, String[][] productos) {  
  
    int posicion;  
  
    posicion = ExisteProducto(codigo, productos);  
  
    if (posicion > -1) {  
        return Double.parseDouble(productos[posicion][4]);  
    }  
  
    return 0;  
}  
  
public static int DescontarStock(  
    String[][] vproductos,  
    String codigo,  
    int cantidad) {  
  
    int posicion;  
    posicion = ExisteProducto(codigo, vproductos);  
    if (posicion == -1) {  
        return -2;  
    }  
    int stock = Integer.parseInt(vproductos[posicion][3]);  
  
    if (stock == 0) {  
        return 0;  
    }  
    if (cantidad > stock) {  
        return -1;  
    }  
    stock = stock - cantidad;  
    vproductos[posicion][3] = String.valueOf(stock);  
    return 1;  
}
```

La función `ObtenerIva` busca un producto mediante su código y obtiene el valor del IVA almacenado en la matriz de productos. Si el producto existe, devuelve el IVA correspondiente; si no existe, devuelve 0.

La función `DescontarStock` se encarga de reducir la cantidad disponible de un producto cuando se realiza una venta. Primero verifica si el producto existe, después revisa si hay stock suficiente y finalmente descuenta la cantidad vendida. También devuelve diferentes valores para indicar si la operación fue correcta o si ocurrió algún error, como producto inexistente o falta de stock.


```
public static void ModificarProducto(String[][] vproductos) throws IOException {  
    String codigo, precio;  
    int posicion;  
    String info = MostrarLista(vproductos);  
    codigo = Leer(info + "\nIntroduce el código del producto a modificar");  
    if (codigo != null) {  
        posicion = ExisteProducto(codigo, vproductos);  
        if (posicion > -1) {  
            String[] vproducto = {  
                vproductos[posicion][0],  
                vproductos[posicion][1],  
                vproductos[posicion][2],  
                vproductos[posicion][3],  
                vproductos[posicion][4]  
            };  
            precio = LeerValidado(  
                "\nIntroduce el precio de " + MostrarProducto(vproducto) + " ",  
                2  
            );  
            vproductos[posicion][2] = precio;  
            System.out.println("precio actualizado");  
        } else {  
            System.out.println("no existe el código");  
        }  
    } else {  
        System.out.println("dato nulo");  
    }  
}
```

La función `ModificarProducto` permite cambiar el precio de un producto registrado en el sistema. Primero muestra la lista de productos y solicita el código del producto que se desea modificar. Después verifica si el producto existe y, en caso correcto, pide el nuevo precio validando que sea numérico. Finalmente actualiza el precio en la matriz de productos y muestra un mensaje confirmando que el precio fue actualizado. Si el código no existe o el dato es nulo, muestra un mensaje de error.

```
public static void MenuProductos (String[][] vproductos) throws IOException
{
    String[] datosmenuproductos = { "1.-Modificar ", "2.-Listado ", "3.-Salida " };
    String opcion = "0";
    do
    {
        opcion = DesplegarMenu("Opciones de Productos",
            datosmenuproductos);
        if (opcion == null)
            System.out.println("opcion incorrecta ");
        else
            switch (opcion)
            {
                case "1": ModificarProducto(vproductos); break;
                case "2": System.out.println(MostrarLista(vproductos)); break;

                case "3":
                    System.out.println("Salida del Sistema "); break;
                default: System.out.println("No existe esta opcion "); break;
            }
        while (opcion.compareTo("3") != 0);
    }

    /**
     * Crea e inicializa el arreglo para las ventas.
     *
     * @return Una matriz bidimensional que representa las ventas.
     */
    public static String[][] CrearVenta()
    {
        return new String[tamventas][5];
    }
}
```

La función MenuProductos muestra el menú de opciones relacionadas con los productos del sistema. Permite al usuario modificar precios, listar todos los productos o salir del menú. Utiliza un ciclo para mantener el menú activo hasta que el usuario seleccione la opción de salida y también valida que las opciones ingresadas sean correctas.

La función CrearVenta crea e inicializa la matriz donde se almacenarán las ventas realizadas en el sistema. Esta matriz guarda la información de los productos vendidos durante las operaciones del punto de venta.

```
public static String UltimoTicket(int pos, String[][] mventa) {
    String idticket = "000"; // Inicializa el ID del ticket como "000" por defecto
    if (pos > -1) { // Verifica si la posición es válida
        idticket = mventa[pos][0]; // Asigna el ID del ticket desde el arreglo de ventas
    }
    return idticket; // Devuelve el ID del último ticket
}

/**
 * Crea e inicializa el arreglo para los tickets.
 *
 * @return Una matriz bidimensional que representa los tickets.
 */
public static String[][] CrearTicket() {
    return new String[20][4];
}

/**
 * Verifica si un código de producto existe en el ticket.
 *
 * @param mticket La matriz que representa el ticket.
 * @param codigo El código del producto a buscar.
 * @return La posición del producto en el ticket si existe, o -1 si no se encuentra.
 */
public static int ExisteTicketCodigo(String[][] mticket, String codigo) {
    int enc = -1; // Inicializa la variable de posición como -1 (no encontrado)
    int pos = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket
    System.out.println(" buscando " + codigo + " tamaño arreglo " + pos);
    for (int ciclo = 0; ciclo <= pos; ciclo++) {
        // Compara el código del producto con el código proporcionado
        if (mticket[ciclo][0].compareTo(codigo.trim()) == 0) {
            enc = ciclo; // Asigna la posición donde se encontró el producto
            return enc; // Retorna la posición encontrada
        }
    }
    return enc; // Retorna -1 si no se encuentra el producto
}
```

La función UltimoTicket obtiene el identificador del último ticket registrado en la matriz de ventas. Si existe una posición válida, toma el ID del ticket almacenado; de lo contrario devuelve "000" como valor inicial.

La función CrearTicket crea e inicializa la matriz donde se almacenará la información de los tickets de venta. Esta matriz guarda los productos agregados en cada compra realizada.

La función ExisteTicketCodigo verifica si un producto ya existe dentro del ticket actual. Recorre los productos registrados en el ticket y compara los códigos para encontrar coincidencias. Si encuentra el producto devuelve la posición donde está almacenado y, si no existe, devuelve -1.

```
public static boolean InsertarProductoTicket(String[][] mticket, String[] datos, int tamticket) {
    boolean sucedio = true; // Indica si la operación fue exitosa
    int posticket = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket
    int enc = ExisteTicketCodigo(mticket, datos[0]); // Verifica si el producto ya existe en el ticket

    if (posticket < tamticket) { // Verifica si hay espacio en el ticket
        if (enc > -1) { // Si el producto ya existe en el ticket
            int cantidadactual = Integer.parseInt(mticket[enc][3]); // Obtiene la cantidad actual del producto
            mticket[enc][3] = String.valueOf(cantidadactual + 1); // Incrementa la cantidad
        } else { // Si el producto no existe en el ticket
            posticket++; // Incrementa la posición del ticket
            mticket[posticket][0] = datos[0]; // Asigna el código del producto
            mticket[posticket][1] = datos[1]; // Asigna el nombre del producto
            mticket[posticket][2] = datos[2]; // Asigna el precio del producto
            mticket[posticket][3] = datos[3]; // Asigna la cantidad del producto
            System.out.println("Insertando: No existe el producto en el ticket");
        }
    } else {
        sucedio = false; // Indica que no se pudo insertar el producto porque el ticket está lleno
    }
    return sucedio; // Devuelve el resultado de la operación
}

/**
 * Calcula el total por producto multiplicando el precio por la cantidad.
 *
 * @param precio El precio unitario del producto como cadena.
 * @param cantidad la cantidad del producto como cadena.
 * @return El total calculado como una cadena con dos decimales.
 */
public static String TotalProducto(String precio, String cantidad) {
    double total = Double.parseDouble(precio) * Double.parseDouble(cantidad); // Calcula el total
    return String.format("%.2f", total); // Formatea el total con dos decimales
}
```

La función InsertarProductoTicket agrega productos al ticket de venta. Primero verifica si el ticket tiene espacio disponible y después revisa si el producto ya existe dentro del ticket. Si el producto ya fue agregado, aumenta la cantidad; si no existe, crea un nuevo registro con el código, nombre, precio y cantidad del producto. La función devuelve un valor booleano para indicar si la operación se realizó correctamente.

La función TotalProducto calcula el total de un producto multiplicando el precio por la cantidad comprada. Después formatea el resultado para mostrarlo con dos decimales y devolverlo como texto.

```

public static String MostrarProductoTicket(String[][] mticket, int pos) {
    String codigo = RellenarEspacios(mticket[pos][0], 5); // Código del producto
    String producto = RellenarEspacios(mticket[pos][1], 30); // Nombre del producto
    String precio = RellenarEspacios(mticket[pos][2], 10); // Precio del producto
    String cantidad = RellenarEspacios(mticket[pos][3], 5); // Cantidad del producto
    String totalproducto = RellenarEspacios(TotalProducto(mticket[pos][2], mticket[pos][3]), 10); // Total por
                                                                    // producto
    String cadena = codigo.concat(producto + precio + cantidad + totalproducto); // Concatenación de los
                                                                    // detalles
    return cadena; // Devuelve la cadena con los detalles del producto
}

/**
 * Muestra el contenido del ticket de ventas. Recorre el ticket y genera una
 * cadena con los detalles de cada producto.
 *
 * @param mticket la matriz que representa el ticket de ventas.
 * @return Una cadena con el detalle de los productos en el ticket.
 */
public static String MostrarTicket(String[][] mticket) {
    String salida = "";
    int pos = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket
    for (int ciclo = 0; ciclo <= pos; ciclo++)
        salida = salida.concat(MostrarProductoTicket(mticket, ciclo) + "\n"); // Agrega el detalle de cada
                                                                    // producto
    return salida; // Devuelve la cadena con el contenido del ticket
}

/**
 * Calcula el subtotal del ticket sumando el total de cada producto en el
 * ticket.
 *
 * @param mticket la matriz que representa el ticket de ventas.
 * @return El subtotal del ticket como un valor de tipo double.
 */
public static double SubTotalTicket(String[][] mticket) {
    double subtotal = 0;
    int pos = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket
    for (int ciclo = 0; ciclo <= pos; ciclo++)
        subtotal = subtotal + Double.parseDouble(TotalProducto(mticket[ciclo][2], mticket[ciclo][3])); // Suma el total
                                                                    // de cada
                                                                    // producto
    return subtotal; // Devuelve el subtotal calculado
}

```

La función `MostrarProductoTicket` organiza y muestra la información de un producto agregado al ticket de venta. Presenta el código, nombre, precio, cantidad y total del producto en un formato ordenado para facilitar la lectura del ticket.

La función `MostrarTicket` recorre todos los productos agregados al ticket y utiliza la función `MostrarProductoTicket` para mostrar el detalle completo de la compra. Al final devuelve una cadena con toda la información del ticket.

La función `SubTotalTicket` calcula el subtotal de la compra sumando el total de cada producto registrado en el ticket. Recorre todos los productos agregados y devuelve el subtotal acumulado de la venta.

```
public static double IvaTicket(String[][] mticket, String[][] productos) {
    double ivatotal = 0;
    int pos = ObtenerUltimaPosicion(mticket);
    for (int ciclo = 0; ciclo <= pos; ciclo++) {
        String codigo = mticket[ciclo][0];
        double iva = ObtenerIva(codigo, productos);
        if (iva != 0) {
            double precio = Double.parseDouble(mticket[ciclo][2]);
            double cantidad = Double.parseDouble(mticket[ciclo][3]);
            double totalproducto = precio * cantidad;
            ivatotal = ivatotal + ((iva / 100) * totalproducto);
        }
    }
    return ivatotal;
}

/**
 * Calcula el total del ticket, incluyendo el subtotal y el IVA.
 *
 * @param mticket La matriz que representa el ticket de ventas.
 * @return El total del ticket, que incluye el subtotal y el IVA.
 */
public static double TotalTicket(String[][] mticket, String[][] productos) {
    double total = SubTotalTicket(mticket);
    if (total > 0) {
        total = IvaTicket(mticket, productos) + total;
    }
    return total;
}
```

La función IvaTicket calcula el IVA total del ticket recorriendo cada producto vendido, obteniendo el porcentaje de IVA de cada uno y sumándolo al total. La función TotalTicket obtiene primero el subtotal del ticket y después le agrega el IVA calculado para regresar el total final de la venta.

```

1 public static String MostrarTicketVenta(
2     String[][] mticket,
3     String idticket,
4     String fecha,
5     String[][] productos) {
6     String salida = "";
7     String subtotal = String.format("%.2f",
8         SubTotalTicket(mticket));
9     String iva = String.format("%.2f",
10        IvaTicket(mticket, productos));
11     String total = String.format("%.2f",
12        TotalTicket(mticket, productos));
13     salida = "Fecha " + fecha + "Ticket No." + idticket;
14     salida = salida + "\n" + MostrarTicket(mticket);
15     salida = salida + "\n \n El total sin iva " + subtotal;
16     salida = salida + "\n el iva total es " + iva;
17     salida = salida + "\n el total de la venta fue " + total;
18     return salida;
19 }
20
21 /**
22  * Muestra la lista de productos disponibles para la venta. Recorre el
23  * inventario de productos y verifica si hay existencia disponible (cantidad
24  * mayor a 0). Si el producto está disponible, lo agrega a la lista de salida.
25  *
26  * @param vproductos la matriz que representa el inventario de productos.
27  * @return Una cadena con la lista de productos disponibles para la venta.
28  */
29 public static String MostrarListaProductosVenta(String[][] vproductos) {
30     String salida = "";
31     for (int ciclo = 0; ciclo < vproductos.length; ciclo++) {
32         int existencia = Integer.parseInt(vproductos[ciclo][3]); // Obtiene la cantidad disponible del producto
33         if (existencia > 0) {
34             String[] vproducto = vproductos[ciclo].clone(); // Clona los datos del producto
35             String cadena = MostrarProducto(vproducto); // Genera la cadena con los datos del producto
36             salida = salida.concat(cadena + "\n"); // Agrega el producto a la lista de salida
37         }
38     }
39     return salida; // Devuelve la lista de productos disponibles
40 }

```

La función `MostrarTicketVenta` genera y muestra el ticket completo de una venta. Presenta la fecha, número de ticket, lista de productos comprados, subtotal, IVA y total final de la compra, todo con formato ordenado para facilitar su lectura.

La función `MostrarListaProductosVenta` muestra únicamente los productos que tienen existencia disponible en el inventario. Recorre la lista de productos y agrega a la salida solo aquellos cuya cantidad en stock es mayor a cero.


```
public static void CapturaVentaProducto(String[][] mticket, String[][] mproductos, String idticket, int tamticket)
    throws IOException {
    String codigo, info;
    info = MostrarListaProductosVenta(mproductos);
    codigo = Leer(info + "\nIntroduce el codigo del producto");
    if (codigo != null) {
        int posp = ExisteProducto(codigo.trim(), mproductos);
        if (posp > -1) {
            String[] producto = mproductos[posp].clone();
            int resultado = DescontarStock(mproductos, codigo, 1);
            if (resultado == 1) {
                System.out.println(MostrarProducto(producto));

                String[] venta = new String[4];
                venta[0] = mproductos[posp][0];
                venta[1] = mproductos[posp][1];
                venta[2] = mproductos[posp][2];
                venta[3] = "1";
                if (!InsertarProductoTicket(mticket, venta, tamticket)) {
                    System.out.println("el Arreglo esta lleno");
                }
            } else if (resultado == 0) {
                System.out.println("no hay productos para venta");
            } else if (resultado == -1) {
                System.out.println("la cantidad es mayor al stock");
            } else if (resultado == -2) {
                System.out.println("el codigo no existe no se puede agregar");
            }
        } else {
            System.out.println("el codigo no existe no se puede agregar");
        }
    } else {
        System.out.println("dato nulo");
    }
}
```

La función CapturaVentaProducto se encarga de registrar un producto dentro de una venta. Primero muestra la lista de productos disponibles y solicita el código del producto que el cliente desea comprar. Después verifica si el producto existe y si hay suficiente stock disponible. Si la validación es correcta, descuenta una unidad del inventario y agrega el producto al ticket de venta. También muestra mensajes de error cuando el producto no existe, no hay stock suficiente o el ticket está lleno.


```

public static void RemoveProductoTicket(String[][] mticket, int pos) {
    int tam = ObtenerUltimaPosicion(mticket);

    if (tam > pos) {
        for (int i = pos; i < tam; i++) {
            mticket[i][0] = mticket[i + 1][0];
            mticket[i][1] = mticket[i + 1][1];
            mticket[i][2] = mticket[i + 1][2];
            mticket[i][3] = mticket[i + 1][3];
        }

        mticket[tam][0] = null;
        mticket[tam][1] = null;
        mticket[tam][2] = null;
        mticket[tam][3] = null;
    }
}

/**
 * Elimina un producto del ticket. Si la cantidad del producto es mayor a 1,
 * simplemente decrementa la cantidad. Si la cantidad es 1, elimina el producto
 * completamente del ticket.
 *
 * @param mticket la matriz que representa el ticket de ventas.
 * @param pos la posición del producto en el ticket que se desea eliminar.
 */
public static void EliminarProductoTicket(String[][] mticket, int pos) {
    int cantidad = Integer.parseInt(mticket[pos][3]); // Obtiene la cantidad del producto en el ticket
    if (cantidad > 1) {
        mticket[pos][3] = String.valueOf(cantidad - 1); // Decrementa la cantidad si es mayor a 1
    } else {
        RemoveProductoTicket(mticket, pos); // Elimina el producto completamente si la cantidad es 1
    }
}

```

La función RemoveProductoTicket elimina un producto completo del ticket de venta. Para hacerlo, mueve las posiciones de los productos siguientes hacia arriba y limpia la última posición de la matriz para evitar datos repetidos o vacíos incorrectos.

La función EliminarProductoTicket se encarga de quitar productos del ticket. Si la cantidad del producto es mayor a uno, solo reduce la cantidad en una unidad; si la cantidad es uno, elimina completamente el producto utilizando la función RemoveProductoTicket.

```

public static void Eliminar(String[][] mticket, String[][] mproductos) throws IOException {
    String codigo, info;
    info = MostrarTicket(mticket);
    codigo = Leer(info + "\nIntroduce el codigo del producto");
    if (codigo != null) {
        int pos = ExisteTicketCodigo(mticket, codigo);
        if (pos > -1) {
            int posproducto = ExisteProducto(codigo, mproductos);
            String nuevacantidad = String.valueOf(Integer.valueOf(mproductos[posproducto][3]) + 1));
            mproductos[posproducto][3] = nuevacantidad;
            mproductos[posproducto][3] = nuevacantidad;
            EliminarProductoTicket(mticket, pos);
        } else {
            System.out.println("no existe el producto en el ticket");
        }
    } else {
        System.out.println("dato nulo");
    }
}

/**
 * Agrega los productos de un ticket a la matriz de ventas. Recorre el ticket y
 * transfiere cada producto a la matriz de ventas, asignando el ID del ticket a
 * cada registro.
 *
 * @param mticket La matriz que representa el ticket de venta.
 * @param mventa La matriz que representa las ventas realizadas.
 * @param idticket El ID del ticket que se está procesando.
 */
public static void AgregarProductoAVenta(String[][] mticket, String[][] mventa, String idticket) {
    int posventas = ObtenerUltimaPosicion(mventa); // Obtiene la última posición ocupada en ventas
    int posticket = ObtenerUltimaPosicion(mticket); // Obtiene la última posición ocupada en el ticket
    for (int i = 0; i <= posticket; i++) {
        if (mticket[i][0] != null) { // Verifica que la fila del ticket no esté vacía
            posventas++; // Incrementa la posición en ventas
            mventa[posventas][0] = idticket; // Establece el ID del ticket
            mventa[posventas][1] = mticket[i][0]; // Código del producto
            mventa[posventas][2] = mticket[i][1]; // Nombre del producto
            mventa[posventas][3] = mticket[i][2]; // Precio del producto
            mventa[posventas][4] = mticket[i][3]; // Cantidad del producto
        }
    }
}

```

La función Eliminar permite quitar un producto del ticket y devolverlo al inventario. Primero busca el código dentro del ticket; si el producto existe, recupera la posición en la matriz de productos, aumenta en 1 la cantidad del inventario y después elimina el producto del ticket. Si no existe en el ticket, muestra un mensaje de error.

La función AgregarProductoAVenta recorre los productos del ticket y los copia a la matriz de ventas. Para cada producto registrado, guarda el ID del ticket junto con el código, nombre, precio y cantidad, de manera que la venta quede almacenada correctamente.

```
public static void Pagar(String idticket, String[][] mventa, String[][] mticket) {
    int posventas = ObtenerUltimaPosicion(mventa); // Obtiene la última posición ocupada en las ventas
    int post = ObtenerUltimaPosicion(mticket); // Obtiene la última posición ocupada en el ticket

    // Verifica si hay espacio suficiente en el arreglo de ventas
    if ((posventas + post) < 100) {
        AgregarProductoAVenta(mticket, mventa, idticket); // Agrega los productos del ticket a las ventas
    } else {
        System.out.println("Desbordamiento de Memoria de ventas"); // Mensaje de error si no hay espacio
    }
}

/**
 * Realiza la devolución de los productos de un ticket al inventario. Recorre el
 * ticket y, por cada producto, incrementa la cantidad en el inventario.
 */
* @param mticket La matriz que representa el ticket de venta.
* @param mproductos La matriz que representa el inventario de productos.
*/
public static void DevolucionTicket(String[][] mticket, String[][] mproductos) {
    int posmticket = ObtenerUltimaPosicion(mticket); // Obtiene la última posición ocupada en el ticket

    for (int pos = 0; pos <= posmticket; pos++) {
        String codigo = mticket[pos][0]; // Obtiene el código del producto del ticket
        int posp = ExisteProducto(codigo.trim(), mproductos); // Busca el producto en el inventario
        if (posp > -1) { // Si el producto existe en el inventario
            int cant = Integer.parseInt(mticket[pos][3]) + Integer.parseInt(mproductos[posp][3]); // Suma las
                                                                    // cantidades
            mproductos[posp][3] = String.valueOf(cant); // Actualiza la cantidad en el inventario
        }
    }
}

public static void CancelarVenta(String[][] mticket, String[][] mproductos) {
    DevolucionTicket(mticket, mproductos);

    for (int i = 0; i < mticket.length; i++) {
        for (int j = 0; j < mticket[i].length; j++) {
            mticket[i][j] = null;
        }
    }
}
```

La función Pagar guarda los productos del ticket en la matriz de ventas, siempre que haya espacio disponible. La función DevolucionTicket recorre el ticket y regresa las cantidades vendidas al inventario sumándolas al stock de cada producto. La función CancelarVenta llama a DevolucionTicket y luego limpia toda la matriz del ticket para dejarla vacía y lista para una nueva venta.



```
public static void MenuPuntoVenta(String[][] ventas, String idticket, String[][] productos) throws IOException {
    String opcion, membrete;
    boolean pago = false;
    int tamticket = 50;
    String[][] Vticket = new String[tamticket][4];

    idticket = IdTicketSiguiente(idticket);
    String fechadia = Fecha();
    opcion = "";

    do {
        membrete = "Fecha del Día " + fechadia + " Ticket No " + idticket;
        membrete = membrete + "\n-----\n";

        String Tickettexto = MostrarTicket(Vticket).trim();
        if (!Tickettexto.trim().isEmpty()) {
            membrete = membrete + "\n" + Tickettexto + "\n";
        }

        String[] datosmenu = { "1.-Agregar ", "2.-Eliminar ", "3.-Listado ", "4.-Pagar ", "5.-Salida " };
        opcion = DesplegarMenu(membrete + "\n Menu de Punto de Venta", datosmenu);

        if (opcion == null) {
            System.out.println("dato incorrecto introducido");
        } else {
            switch (opcion) {
                case "1":
                    CapturaVentaProducto(Vticket, productos, idticket, tamticket);
                    break;
                case "2":
                    Eliminar(Vticket, productos);
                    break;
                case "3":
                    if (ObtenerUltimaPosicion(Vticket) > -1) {
                        System.out.println(MostrarTicket(Vticket));
                    }
                    break;
                case "4":
                    System.out.println(MostrarTicketVenta(Vticket, idticket, fechadia, productos).trim());
                    Pagar(idticket, ventas, Vticket);
                    pago = true;
                    opcion = "5";
                    break;
            }
        }
    } while (opcion != "5");
}
```

```
        break;
    case "5":
        System.out.println("Salida del Ventas");
        if (!pago) {
            System.out.println("No pago el ticket");
            CancelarVenta(Vticket, productos);
            System.out.println("eliminando tickte " + idticket);
        }
        break;
    default:
        System.out.println("No existe esta opcion");
        break;
}
while (opcion.compareTo("5") != 0);
```

La función MenuPuntoVenta controla todo el proceso de una venta. Primero crea un ticket nuevo, genera el siguiente ID y obtiene la fecha actual. Después muestra un menú con opciones para agregar productos, eliminar productos, mostrar el ticket, pagar o salir.

Cuando el usuario selecciona agregar, llama a la función CapturaVentaProducto para añadir productos al ticket. Si selecciona eliminar, utiliza la función Eliminar para quitar productos del ticket y regresar el stock al inventario. La opción de listado muestra todos los productos agregados al ticket actual.

En la opción pagar, el sistema imprime el ticket completo con subtotal, IVA y total de la venta utilizando MostrarTicketVenta, luego guarda la venta con la función Pagar y marca el ticket como pagado.

Si el usuario sale sin pagar, el sistema muestra un mensaje indicando que el ticket no fue pagado, cancela la venta con CancelarVenta y devuelve los productos al inventario. El ciclo del menú termina cuando el usuario selecciona la opción de salida.

```
public static String MostrarVenta(String[] venta) {  
    String idticket = RellenarEspacios(venta[0], 6); // ID del ticket  
    String codigo = RellenarEspacios(venta[1], 5); // Código del producto  
    String producto = RellenarEspacios(venta[2], 30); // Nombre del producto  
    String precio = RellenarEspacios(venta[3], 10); // Precio del producto  
    String cantidad = RellenarEspacios(venta[4], 10); // Cantidad del producto  
    String cadena = idticket.concat(codigo + producto + precio + cantidad); // Concatenación de los detalles  
    return cadena; // Devuelve la cadena con los detalles de la venta  
}  
  
/**  
 * Muestra la lista de todas las ventas realizadas. Recorre la matriz de ventas  
 * y genera una cadena con los detalles de cada venta.  
 *  
 * @param ventas La matriz que representa las ventas realizadas.  
 * @return Una cadena con el detalle de todas las ventas.  
 */  
public static String MostrarListaVentas(String[][] ventas) {  
    int posventas = ObtenerUltimaPosicion(ventas); // Obtiene la última posición válida en la matriz de ventas  
    String salida = "";  
    for (int ciclo = 0; ciclo <= posventas; ciclo++) {  
        // Crea un arreglo con los datos de una venta específica  
        String[] venta = { ventas[ciclo][0], ventas[ciclo][1], ventas[ciclo][2], ventas[ciclo][3],  
            ventas[ciclo][4] };  
        String cadena = MostrarVenta(venta); // Genera una cadena con los detalles de la venta  
        salida = salida.concat(cadena + "\n"); // Agrega la cadena al resultado final  
    }  
    return salida; // Devuelve la cadena con el detalle de todas las ventas  
}
```

a funcion `MostrarVenta` organiza y muestra la informacion de una venta individual. Toma los datos del ticket, como ID, codigo, producto, precio y cantidad, y los acomoda en una cadena de texto con formato para que se vea ordenado.

La funcion `MostrarListaVentas` recorre toda la matriz de ventas y utiliza `MostrarVenta` para generar una lista completa con todas las ventas realizadas. Al final devuelve el contenido total de las ventas en forma de texto.

```
public static void AgregarStock(String[][] vproductos) throws IOException {
    String codigo, cantidad;
    int posicion;

    String info = MostrarLista(vproductos);
    codigo = LeerValidado(info + "\nIntroduce el codigo del producto a modificar", 1);

    if (codigo != null) {
        posicion = ExisteProducto(codigo, vproductos);

        if (posicion > -1) {

            String[] vproducto = {
                vproductos[posicion][0],
                vproductos[posicion][1],
                vproductos[posicion][2],
                vproductos[posicion][3],
                vproductos[posicion][4]
            };

            cantidad = LeerValidado("\nIntroduce la cantidad de stock a agregar ", 1);

            int stockActual = Integer.parseInt(vproductos[posicion][3]);
            int stockNuevo = Integer.parseInt(cantidad);

            vproductos[posicion][3] = String.valueOf(stockActual + stockNuevo);

            System.out.println("stock actualizado");

        } else {
            System.out.println("no existe el codigo");
        }

    } else {
        System.out.println("dato nulo");
    }
}
```

La funcion AgregarStock permite aumentar la cantidad disponible de un producto en el inventario. Primero muestra la lista de productos y solicita el código del producto que se desea modificar. Después verifica si el producto existe y pide la cantidad de stock que se agregará. Finalmente suma la nueva cantidad al stock actual, actualiza el inventario y muestra un mensaje indicando que el stock fue actualizado correctamente.

```
public static void MenuInventario (String[][] vproductos ) throws IOException
{
    String[] datosmenuinventario = { "1.-Listado ", "2.-Agregar ", "3.-Salida " };
    String opcion = "0";
    do
    {
        opcion = DesplegarMenu("Opciones de Inventarios",datosmenuinventario);
        if (opcion == null)
            System.out.println("opcion incorrecta ");
        else
            switch (opcion)
            {
                case "1": System.out.println(MostrarLista(vproductos)); break;
                case "2": AgregarStock(vproductos); break;
                case "3": System.out.println("Salida del Sistema "); break;
                default: System.out.println("No existe esta opcion "); break;
            }
    }
    while (opcion.compareTo("3") != 0);
}

public static void MenuPrincipal(String[][] vproductos,String[][] vventas) throws IOException
{
    String[] datosmenuprincipal = { "1.-Productos ", "2.-Punto de Venta ", "3.- Inventario", "4.-Ventas", "5.-Salida " };
    String opcion = "0";
    String idticket;
    do
    {
        idticket= ObtenerUltimoValorVentas(vventas);
        opcion = DesplegarMenu("\Menu de Punto de Venta Cafeteria Coffee House\\",
        datosmenuprincipal);
        if (opcion == null)
            System.out.println("opcion incorrecta ");
        else
            switch (opcion)
            {
                //falta adecuar el idticket que revise ventas y si esta vacio sea 000 si no el ultimo del arreglo
                case "1": MenuProductos(vproductos); break;
                case "2": MenuPuntoVenta(vventas,idticket,vproductos); break;
                case "3": MenuInventario(vproductos); break;
                case "4": System.out.println(MostrarListaVentas(vventas)); break;
                case "5":
                    System.out.println("Salida del Sistema "); break;
                default: System.out.println("No existe esta opcion "); break;
            }
    }
}
```

La funcion MenuInventario muestra el menu de inventario y permite listar productos, agregar stock o salir. Usa un ciclo para mantenerse activo hasta que el usuario elija la opcion de salida, y valida si la opcion ingresada es correcta.

La funcion MenuPrincipal muestra el menu principal del sistema de la cafeteria. Desde ahi se accede a productos, punto de venta, inventario y ventas. Tambien obtiene el ultimo ID de venta para generar el ticket siguiente y controla las opciones del sistema hasta que el usuario salga.


```
/**
 * Obtiene el último valor del ID de las ventas registradas. Recorre la matriz
 * de ventas para encontrar la última posición válida y devuelve el valor del ID
 * de esa posición. Si no hay ventas registradas, devuelve "000" como valor
 * predeterminado.
 *
 * @param ventas La matriz que representa las ventas realizadas.
 * @return El último valor del ID de las ventas o "000" si no hay registros.
 */
public static String ObtenerUltimoValorVentas(String[][] ventas) {
    int ultimaposicion = ObtenerUltimaPosicion(ventas); // Obtiene la última posición válida en la matriz de ventas
    String ultimoValor = "000"; // Valor predeterminado si no hay registros
    if (ultimaposicion >= 0) {
        ultimoValor = ventas[ultimaposicion][0]; // Asigna el ID de la última venta registrada
    }
    return ultimoValor; // Devuelve el último valor del ID de las ventas
}

public static void main(String[] args) throws IOException
{
    productos = CargarProductos();
    ventas=CrearVenta();
    MenuPrincipal(productos,ventas);
}
}
```

La función `ObtenerUltimoValorVentas` busca el último ID registrado en la matriz de ventas. Primero obtiene la última posición ocupada y, si existen ventas, guarda el ID de esa posición. Si no hay registros, devuelve "000" como valor predeterminado.

La función `main` es el punto de inicio del programa. Primero carga los productos con `CargarProductos`, después crea la matriz de ventas con `CrearVenta` y finalmente ejecuta el `MenuPrincipal` para iniciar todo el sistema del punto de venta.